Complete Graph Problems & Resources Guide

Hey everyone! I've been working on graph problems for both tech interviews and competitive programming, and I wanted to share a comprehensive guide that covers both aspects. Many resources focus on just one side, but mastering graphs requires understanding both the interview patterns and the deeper algorithmic concepts used in CP.

This guide includes curated problem lists, the best learning resources I've found, and a clear roadmap for both paths.

Understanding the Two Approaches

For Tech Interviews: You need pattern recognition across 10-12 core patterns, clean code, and the ability to explain your approach clearly. About 80-100 well-understood problems is enough to crack most interviews. Focus on BFS, DFS, Topological Sort, Dijkstra, and Union-Find.

For Competitive Programming: You need algorithmic depth, fast implementation with templates, and mastery of advanced algorithms. This requires 300+ problems and participation in contests. Everything from interviews plus advanced topics like SCC, HLD, Max Flow, and more.

Best Learning Resources

Video Resources for Interview Prep

NeetCode - Graph Playlist The best resource for learning interview patterns. Clear explanations with Python code covering 15 essential problems. It takes about 4-5 hours total.

take U forward (Striver) - Graph Series Comprehensive coverage with 50+ problems and all important algorithms. His explanations are detailed with C++/Java code. This is gold for interview prep, around 25-30 hours of content.

Abdul Bari - Graph Algorithms Perfect for understanding concepts deeply. His whiteboard teaching style makes complex algorithms crystal clear. About 15-20 hours of theory.

Video Resources for Competitive Programming

William Fiset - Graph Theory Must-watch for CP. Complete graph theory course covering everything from basics to advanced algorithms like Tarjan's, Network Flow, and Bridges. About 10 hours total.

Errichto - Contest Streams Watch his live problem solving for advanced CP techniques. Great for seeing how top coders approach graph problems in real contests.

Colin Galen Excellent educational content on specific graph topics and optimizations for competitive programming.

Written Resources

For Interviews:

- LeetCode Discuss section for graph problems
- GeeksforGeeks Graph Data Structure section
- Tech Interview Handbook for quick revision

For Competitive Programming:

- CP-Algorithms (cp-algorithms.com) - In-depth explanations with proofs for every algorithm
- USACO Guide - Structured learning path from Bronze to Platinum
- Codeforces Catalog - Problems organized by technique

Books Worth Reading

Cracking the Coding Interview - Chapter 4 covers trees and graphs, essential for FAANG prep

Competitive Programming 4 by Steven Halim - The bible for CP, extensive graph coverage across multiple chapters

Competitive Programmer's Handbook by Antti Laaksonen - Excellent practical coverage, free PDF available at cses.fi/book

---

Interview Problems (80 Total)

Phase 1: Foundations

Graph Traversal:

1. Number of Islands
2. Max Area of Island
3. Clone Graph
4. Number of Connected Components
5. Pacific Atlantic Water Flow
6. Surrounded Regions
7. Walls and Gates
8. Rotting Oranges
9. 01 Matrix
10. As Far from Land as Possible

Matrix Problems:

11. Number of Closed Islands

12. Number of Enclaves

13. Count Sub Islands

14. Island Perimeter

15. Shortest Path in Binary Matrix

Basic Validation:

16. Graph Valid Tree

17. Find if Path Exists in Graph

18. All Paths From Source to Target

19. Find the Town Judge

20. Keys and Rooms

Phase 2: Cycle Detection & Topological Sort

21. Course Schedule
22. Course Schedule II
23. Redundant Connection

24. Redundant Connection II
25. Find Eventual Safe States
26. Alien Dictionary
27. Sequence Reconstruction
28. Minimum Height Trees
29. Sort Items by Groups Respecting Dependencies
30. Parallel Courses
31. Parallel Courses II
32. Build a Matrix With Conditions
33. Largest Color Value in a Directed Graph
34. Longest Cycle in a Graph
35. Longest Path With Different Adjacent Characters

Phase 3: Shortest Path

BFS Shortest Path:

36. Word Ladder

37. Word Ladder II

38. Open the Lock

39. Minimum Knight Moves

40. Shortest Bridge

Dijkstra's Algorithm:

41. Network Delay Time

42. Path with Maximum Probability

43. Cheapest Flights Within K Stops

44. Path with Minimum Effort

45. Swim in Rising Water

46. Minimum Cost to Reach Destination in Time

47. Minimum Weighted Subgraph With Required Paths

48. Minimum Obstacle Removal to Reach Corner

49. Minimum Cost of a Path With Special Roads

50. Find the City With Smallest Number of Neighbors

Phase 4: Union-Find

51. Number of Provinces
52. Redundant Connection
53. Most Stones Removed with Same Row or Column
54. Accounts Merge
55. Smallest String With Swaps
56. Satisfiability of Equality Equations
57. Regions Cut By Slashes
58. Number of Islands II
59. Graph Connectivity With Threshold
60. Minimize Malware Spread
61. Minimize Malware Spread II
62. Checking Existence of Edge Length Limited Paths
63. Last Day Where You Can Still Cross
64. Maximum Number of Fish in a Grid
65. Count Unreachable Pairs of Nodes

Phase 5: Advanced Topics

66. Min Cost to Connect All Points

67. Connecting Cities With Minimum Cost

68. Optimize Water Distribution in a Village

69. Critical Connections in a Network

70. Find Critical and Pseudo-Critical Edges in MST

71. Is Graph Bipartite?

72. Possible Bipartition

73. Divide Nodes Into the Maximum Number of Groups

74. Reconstruct Itinerary

75. Bus Routes

76. Shortest Path Visiting All Nodes

77. Minimum Number of Days to Disconnect Island

78. Reachable Nodes In Subdivided Graph

79. Shortest Path to Get All Keys

80. Sliding Puzzle

Competitive Programming Problems

CSES Problem Set (Must Solve All 40)

These problems are exceptionally well-designed and cover every important graph algorithm. Solve them in order.

Basic Graphs:

1. Counting Rooms
2. Labyrinth
3. Building Roads
4. Message Route
5. Building Teams
6. Round Trip
7. Monsters
8. Shortest Routes I
9. Shortest Routes II
10. High Score

Advanced Paths:

11. Flight Discount

12. Cycle Finding

13. Flight Routes

14. Round Trip II

15. Course Schedule

16. Longest Flight Route

17. Game Routes

18. Investigation

19. Planet Queries I
20. Planet Queries II

Tree Algorithms:

21. Subordinates
22. Tree Matching
23. Tree Diameter
24. Tree Distances I
25. Tree Distances II
26. Company Queries I
27. Company Queries II
28. Distance Queries
29. Counting Paths
30. Subtree Queries

Advanced Topics:

31. Path Queries
32. Path Queries II
33. Distinct Routes
34. Police Chase
35. School Dance
36. Coin Collector
37. Mail Delivery
38. De Bruijn Sequence
39. Teleporters Path
40. Hamiltonian Flights

Codeforces Problems by Rating

Rating 1200-1400 (Beginner CP):

1. Kefa and Park
2. Learning Languages
3. DZY Loves Chemistry
4. Greg and Graph
5. King's Path

Rating 1400-1600 (Intermediate):

6. News Distribution

7. Fox And Names

8. Dijkstra?

9. Road Construction

10. Checkposts

Rating 1600-1800 (Advanced):

11. Xenia and Tree

12. Edges in MST

13. Beautiful Graph

14. Minimum spanning tree for each edge

15. Scheme

Rating 1800-2000 (Expert):

16. The Door Problem

17. Digit Tree

18. Tourist Reform

19. Ideal Path

20. Bosses

AtCoder Problems

These contests have excellent problem quality. Focus on problems D, E, F from ABC contests.

1. ABC139D - ModSum
2. ABC160D - Line++
3. ABC168D - Double Dots
4. ABC170D - Not Divisible
5. ABC213D - Takahashi Tour

SPOJ Classic Problems

1. SHPATH - The Shortest Path
2. PRIM - Prim Algorithm
3. SUBMERGE - Submerge
4. LCA - Lowest Common Ancestor
5. QTREE - Query on a tree
6. QTREE2 - Query on a tree II
7. FASTFLOW - Fast Maximum Flow

Learning Timeline

For Interview Preparation (8 weeks)

Week 1: Graph Traversal Solve Phase 1 problems (20 problems). Master BFS and DFS implementations. Understand when to use each. By the end, you should be able to code both from memory.

Week 2: Cycle Detection & Topological Sort Solve Phase 2 problems (15 problems). Learn cycle detection for both directed and undirected graphs. Master Kahn's algorithm for topological sort. Understand the applications of topological sort.

Week 3: Shortest Path Solve Phase 3 problems (15 problems). Master BFS for unweighted graphs. Implement Dijkstra's algorithm from scratch. Understand priority queue operations clearly.

Week 4: Union-Find Solve Phase 4 problems (15 problems). Implement DSU with path compression and union by rank. This data structure is elegant and appears frequently in interviews.

Week 5: Advanced Topics Solve Phase 5 problems (15 problems). Learn MST using Kruskal's algorithm. Understand bridges and articulation points. Study bipartite graph problems.

Weeks 6-8: Practice & Review Solve 20-30 mixed problems. Do mock interviews. Time yourself on medium problems (target 25-30 minutes). Review problems you struggled with.

For Competitive Programming (6 months)

Month 1: Foundations Complete all basic CSES problems (1-10). Solve Codeforces problems rated 1200-1400. Focus on clean DFS and BFS implementations. Target 50-60 problems this month.

Month 2: Shortest Paths Complete CSES shortest path section. Learn Dijkstra, Bellman-Ford, and Floyd-Warshall. Solve Codeforces 1400-1600 rated problems. Target 40-50 problems.

Month 3: Core Algorithms Study topological sort, cycle detection, DSU, and bipartite checking. Complete CSES problems 11-20. Solve Codeforces 1400-1600 problems. Target 50-60 problems.

Month 4: Trees & SCC Master tree DP and basic LCA. Learn Kosaraju's and Tarjan's algorithms for SCC. Complete CSES tree section. Solve Codeforces 1600-1800 problems. Target 40-50 problems.

Month 5: Advanced Trees Study Heavy-Light Decomposition and Centroid Decomposition. Learn advanced LCA techniques. Complete remaining CSES problems. Target 30-40 problems.

Month 6: Advanced Topics Learn Dinic's algorithm for max flow, 2-SAT, and advanced applications. Solve Codeforces 1800+ problems. Participate in virtual contests regularly. Target 30-40 problems.

Study Approach

For Interviews

Start with one pattern and solve 8-10 problems before moving to the next. When you encounter a problem, spend 20-30 minutes trying to solve it yourself. If stuck, look at hints or the approach (not the full solution), then implement yourself. After solving, review your code and optimize if possible.

Use spaced repetition: solve a problem today, revisit it after 3 days, then after a week, then after a month. This helps with long-term retention.

Focus on explaining your thought process clearly. Practice talking through your approach out loud.

For Competitive Programming

Solve problems rated 200 points above your current rating. When you achieve 80% success rate, move up. Participate in 2-3 virtual contests per week. Always upsolve problems you couldn't solve during contests.

Read editorials even for problems you solved - you'll often learn better approaches. Build a template library as you learn algorithms. Practice implementing algorithms from scratch without looking at references.

Join contest discussions on Codeforces to learn from others. Don't spend more than 2 hours stuck on a single problem - read the editorial and implement it yourself.

Final Thoughts

For interviews, quality matters more than quantity. Deeply understanding 80-100 problems across all patterns is better than superficially solving 500 problems. Focus on pattern recognition and clean implementation.

For competitive programming, consistency is key. Solve problems daily, participate in contests regularly, and gradually increase difficulty. Your rating will improve steadily with consistent practice.

Both paths are rewarding. Interview preparation helps you crack top tech companies. Competitive programming makes you a better programmer overall and opens doors to international competitions.

Choose your path based on your goals, but remember that the skills are complementary. Good luck with your graph journey!
